

React Native Interview

Questions – Simple Answers

(Say This in Interview)

Plain English. Just Question + Answer. Read it like you're explaining to a friend, not reading a textbook.

Table of Contents

Basics: [1](#) · [2](#) · [3](#) · [4](#) · [5](#) · [6](#) · [7](#) · [8](#)

Hooks: [9](#) · [10](#) · [11](#) · [12](#) · [13](#) · [14](#) · [15](#) · [16](#) · [17](#)

Components & UI: [18](#) · [19](#) · [20](#) · [21](#) · [22](#) · [23](#)

Navigation: [24](#) · [25](#) · [26](#)

Redux & State: [27](#) · [28](#) · [29](#) · [30](#)

API & Networking: [31](#) · [32](#)

Storage: [33](#) · [34](#) · [35](#)

Lists & Performance: [36](#) · [37](#) · [38](#)

Native, Firebase, Permissions: [39](#) · [40](#) · [41](#) · [42](#)

Lifecycle & Error Handling: [43](#) · [44](#) · [45](#)

Animation: [46](#)

Libraries, Styling, Testing, Release: [47](#) · [48](#) · [49](#) · [50](#) · [51](#) · [52](#)

1. What is React Native?

Answer: It's a way to build mobile apps for Android and iOS using JavaScript and React. The good part is, it doesn't show a website inside the app — it actually creates real native screens, like a normal Android or iOS app. So it feels fast, not like a webpage.

2. What is the React Native Architecture? (Bridge, New Architecture, JSI, Fabric, TurboModules)

Answer: Earlier, JavaScript code and native code (Java/Kotlin/Swift) used to talk to each other through something called the "Bridge." It was a bit slow because everything had to be converted to JSON first. Now there's a "New Architecture" that lets JS talk to native code directly and faster, using something called JSI. Inside this New Architecture, "Fabric" handles how the screen draws UI faster, and "TurboModules" loads native code only when it's actually needed, so the app opens faster.

In short, if asked simply: Old way = slow bridge with JSON. New way = direct, faster communication using JSI.

3. What is Hermes?

Answer: Hermes is a JavaScript engine made for React Native. It makes the app start faster, use less memory, and the app size becomes smaller. Most new React Native projects have it turned on by default — you don't need to do anything extra.

4. Functional Components vs Class Components

Answer: Class components are the old style — they use `this` and special lifecycle methods. Functional components are the new style — they're just plain functions, and we use hooks like `useState` and `useEffect` to manage data. Today, almost everyone writes functional components because they're shorter and easier to work with.

```
function Counter() {  
  const [count, setCount] = useState(0);  
  return <Button title={` ${count}`} />  
}
```

```
onPress={() => setCount(count + 1)} />;  
}
```

5. Props vs State

Answer: Props is the data that comes from the parent component — you can't change it inside the child.

State is the data that lives inside the component itself, and it can change. When state changes, the screen re-renders automatically.

6. What is JSX?

Answer: JSX is the syntax that lets us write HTML-like code inside JavaScript, like `<View>`

`<Text>Hello</Text></View>` . The browser/native side doesn't understand this directly — it gets converted (compiled) into normal JavaScript function calls behind the scenes. It just makes writing UI code much easier to read.

7. Expo vs React Native CLI

Answer: Expo is a set of tools built on top of React Native that makes it easier to start a project — you don't need to touch native Android/iOS code, and a lot

of features (camera, location, notifications) work out of the box. React Native CLI gives you full control over the native code, which you need when you have to add custom native modules or libraries that Expo doesn't support. Many teams start with Expo and only move to a "bare workflow" or plain CLI if they need deeper native control.

8. What is Metro Bundler?

Answer: Metro is the tool that takes all your JavaScript/TypeScript files, combines them into one bundle, and serves it to the app. It's basically React Native's version of Webpack. It also gives you fast refresh, so you see your changes immediately while developing.

9. useState

Answer: This hook is used to store and update a value inside a component. It gives you two things — the current value, and a function to change it.

```
const [count, setCount] = useState(0);
```

10. useEffect

Answer: This hook runs some code after the screen renders — mostly used for API calls, or anything that needs to happen automatically. If you give it an empty array `[]`, it runs only once when the screen loads. If you give it a value inside the array, it runs again whenever that value changes.

```
useEffect(() => {  
  fetchUserData();  
}, []); // runs once
```

11. `useLayoutEffect`

Answer: This is almost the same as `useEffect`, but it runs at a different time — it runs right after the UI is updated, but before the user actually sees it on screen. `useEffect` runs a little after, once the screen is already shown. We use `useLayoutEffect` only when we need to measure something on the screen (like a component's height or width) and adjust it before the user notices any flicker. In React Native, we use this much less often than `useEffect`, because most of the time `useEffect` is good enough.

```
useLayoutEffect(() => {  
  boxRef.current?.measure((x, y, width,  
    height) => {  
    setHeight(height); // set before user  
    sees it  
  })  
})
```

```
});  
}, []);
```

12. useRef

Answer: This hook gives you a value that stays the same between renders, but changing it does not refresh the screen. It's mostly used to directly access something like a `TextInput` (for example, to focus it), or to store a value you don't want to trigger a re-render for.

```
const inputRef = useRef(null);  
inputRef.current?.focus();
```

13. useMemo

Answer: This hook saves the result of a heavy calculation, so it doesn't run again and again on every render. It only re-calculates when the values it depends on actually change.

```
const filteredList = useMemo(() =>  
  list.filter(item => item.active), [list]);
```

14. useCallback

Answer: This is similar to `useMemo`, but instead of saving a value, it saves a function — so the same function is reused instead of creating a new one every time the component renders. This is mostly useful when you pass that function to a child component.

```
const handlePress = useCallback(() =>
  console.log("pressed"), []);
```

15. useReducer

Answer: This is like `useState`, but used when the state logic is more complex — when many things need to be updated together based on different actions. You send an "action," and a function decides what the new state should be. It works almost like a mini version of Redux.

16. useEffect

Answer: This comes from React Navigation. It runs code every time you come back to a screen, not just the first time it loads. This is different from `useEffect`, because in React Native, screens often stay alive in the background when you navigate away, so plain `useEffect` won't run again on its own.

17. Custom Hooks

Answer: A custom hook is just our own function that starts with the word "use," and it can use other hooks inside it. We make custom hooks so we don't have to repeat the same logic in many components.

```
function useDebounce(value, delay) {
  const [debounced, setDebounced] =
    useState(value);
  useEffect(() => {
    const timer = setTimeout(() =>
      setDebounced(value), delay);
    return () => clearTimeout(timer);
  }, [value, delay]);
  return debounced;
}
```

18. Controlled vs Uncontrolled Components

Answer: A controlled component's value is fully managed by state — whatever is in state is shown, and every change updates the state (`value` + `onChangeText`). An uncontrolled component manages its own value internally, and we only read it when needed using a `ref`, without updating state on every keystroke. Most forms in React Native use

controlled inputs because we usually need live validation.

19. `TouchableOpacity` vs `TouchableHighlight` vs `Pressable` vs `TouchableWithoutFeedback`

Answer: These are all used to make something tappable. `TouchableOpacity` makes the item fade a little when pressed. `TouchableHighlight` adds a highlight/background color behind it when pressed. `TouchableWithoutFeedback` shows no visual feedback at all when pressed. `Pressable` is the newest and most flexible one – it can handle press, long press, and gives you full control to style based on press state, so most new code uses `Pressable` now.

20. Why use `StyleSheet.create` instead of a plain style object?

Answer: `StyleSheet.create` is a little optimized – React Native can send the styles to the native side once and reuse them by reference, instead of recreating a new style object on every render. It also gives better error messages if you make a typo in a style property, and most teams use it just to keep styles organized and consistent.

21. SafeAreaView

Answer: `SafeAreaView` makes sure your content doesn't get hidden behind things like the phone's notch, status bar, or the home indicator at the bottom (especially on iPhones). It automatically adds padding so your UI stays inside the visible safe area of the screen.

22. KeyboardAvoidingView

Answer: This component is used so that the keyboard doesn't cover your input fields when it pops up. It automatically pushes the content up so the user can still see what they're typing, especially useful on forms or chat screens.

23. Why do list items need a unique key ?

Answer: React uses the `key` to figure out which items changed, were added, or were removed, so it can update only what's needed instead of re-rendering the whole list. If you don't give a proper unique key (or use the index as a key on a list that can reorder), you can get bugs like wrong data showing up in the wrong row, especially when items are added, removed, or reordered.

24. Stack, Tab, and Drawer Navigation

Answer: Stack navigation is when screens open one on top of another, like a stack of cards — and you can go back. Tab navigation is the bottom tabs you see in most apps, like Home, Cart, Profile. Drawer navigation is the side menu that slides in from the left or right. Usually apps use a mix — for example, tabs at the bottom, and each tab has its own stack inside.

25. Deep Linking

Answer: Deep linking means opening the app directly on a specific screen using a link — for example, tapping a notification opens the Order Details page directly, instead of opening the home screen first.

26. `useNavigation` & `useRoute` / Passing data between screens

Answer: `useNavigation` is a hook that gives you the navigation object so you can move to another screen, like `navigation.navigate("Details")`. `useRoute` gives you access to the params that were passed to the current screen. To send data to another screen, you pass it as a second argument while navigating, and read it on the other side using `route.params`.

```
navigation.navigate("Details", { productId:
5 });

// On Details screen
const route = useRoute();
const { productId } = route.params;
```

27. What are the core parts of Redux? (Store, Action, Reducer, Dispatch, Provider)

Answer: Redux has a few simple parts. The **Store** is where all your app's data (state) is kept in one place. An **Action** is just a plain object that says what happened, like `{ type: "ADD_TO_CART" }`. A

Reducer is a function that takes the current state and the action, and returns the new state. **Dispatch** is how you send an action to the store. The **Provider** is a component that wraps your whole app so that every component can access the store. So the simple flow is: you dispatch an action → the reducer updates the state → the store holds the new state → your component re-renders with the new data.

28. Redux Toolkit (createSlice, configureStore, useSelector, useDispatch)

Answer: Redux Toolkit is the modern, easier way to write Redux — it removes a lot of repetitive code.

`createSlice` lets you write your state, actions, and reducer all together in one place. `configureStore` sets up the store for you with good settings already applied. `useSelector` is used inside a component to read data from the store. `useDispatch` is used to send an action to update the store.

```
const slice = createSlice({
  name: "counter",
  initialState: { value: 0 },
  reducers: { increment: (state) => {
    state.value += 1; } },
});
const count = useSelector((state) =>
state.counter.value);
const dispatch = useDispatch();
dispatch(slice.actions.increment());
```

29. RTK Query

Answer: This is a tool inside Redux Toolkit that handles API calls for you. It automatically gives you loading, error, and data states, and it also handles caching, so you don't need to write `useEffect` + `Axios` + manual loading state every time you need to call an API.

```
const { data, isLoading } =  
useGetUserQuery(1);
```

30. Context API

Answer: This is a built-in React feature that lets you share data across many components without passing it down manually through every single level (called "prop drilling"). It's good for things that don't change very often, like theme (dark/light mode) or the logged-in user's info.

31. Axios and Interceptors

Answer: Axios is a library used to call APIs — GET, POST, PUT, DELETE. Interceptors let you run some code automatically before every request goes out, or after every response comes back. For example, you can automatically attach the login token to every API call, or automatically log the user out if you get a 401 error — without writing that code again and again in every API call.

```
api.interceptors.request.use((config) => {  
  config.headers.Authorization = `Bearer  
  ${token}`;  
});
```

```
    return config;
  });
```

32. Handling No Internet / NetInfo

Answer: We use a library like `@react-native-community/netinfo` to check if the device is connected to the internet. We usually listen for connection changes and show a banner like "No internet connection" if the user goes offline, and maybe retry failed API calls automatically once the connection comes back.

```
NetInfo.addEventListener(state => {
  setIsConnected(state.isConnected);
});
```

33. AsyncStorage vs Secure Storage

Answer: AsyncStorage is used to save simple data like a key and value on the phone, but it is not encrypted – so don't save sensitive things like passwords or tokens in it. For sensitive data, we use Secure Storage (like Keychain on iOS or Keystore on Android), which keeps the data encrypted and safe.

34. Offline Storage

Answer: Offline storage means saving data on the phone so the app can still work even without internet. We usually do this by saving API data locally (using AsyncStorage, SQLite, or a database like Realm/WatermelonDB), and when the internet comes back, we sync the local data with the server.

How it's usually done: Save data locally first → show it to the user immediately → when internet is available, call the API and update both the server and the local copy.

35. SQLite in React Native

Answer: SQLite is a small database that can run directly inside the phone, used when we have a lot of structured data to store locally – more than what AsyncStorage is good for. We usually use a library like `react-native-sqlite-storage` or `expo-sqlite` to create tables and run normal SQL queries (like `SELECT`, `INSERT`, `UPDATE`) right inside the app.

```
db.transaction((tx) => {
  tx.executeSql(
    "INSERT INTO orders (id, name) VALUES",
    (?, ?)",
    [1, "Pizza"]
  )
})
```

```
);  
});
```

When to use SQLite instead of AsyncStorage: When you have a lot of data, need to search/filter it, or need relationships between data (like orders belonging to a user) — AsyncStorage is just simple key-value, not good for that.

36. FlatList and FlatList Optimization

Answer: FlatList is used to show long lists efficiently — it only renders the items currently visible on the screen, not the whole list at once. This is why we use FlatList instead of ScrollView for big lists. To make it even faster, we use `keyExtractor` for unique keys, `getItemLayout` if all items are the same size, and wrap each list item in `React.memo` so it doesn't re-render unless its own data changes.

37. Performance Optimization

Answer: Some common ways to make a React Native app faster: use `React.memo` so a component doesn't re-render unless its props change, use `useMemo` / `useCallback` to avoid doing the same work again and again, load heavy screens only when

needed (lazy loading), and optimize images (correct size, caching) so they don't eat up memory.

38. React.memo vs PureComponent vs shouldComponentUpdate

Answer: All three are used to stop unnecessary re-renders. `React.memo` is used for functional components — it skips re-rendering if the props haven't changed. `PureComponent` is the class component version of the same idea. `shouldComponentUpdate` is a manual lifecycle method (only in class components) where you write your own logic to decide if the component should re-render or not, giving you full control.

39. Native Modules / Bridge

Answer: Sometimes React Native doesn't have a ready-made way to do something — like using a special hardware feature. In that case, we write some native Android/iOS code ourselves and connect it to JavaScript using something called a Native Module, so we can call that native code from our JS code.

40. Firebase Push Notifications (FCM)

Answer: FCM is used to send push notifications to the app. The app behaves differently depending on its state: when the app is **open** (foreground), the notification doesn't show automatically, so we usually show our own custom popup. When the app is **in the background**, the notification shows automatically by the system, and tapping it can take the user to a specific screen. When the app is **completely closed** (killed), we need a separate piece of code to handle what happens when the user taps the notification and opens the app.

41. Permissions (Camera, Location, Storage)

Answer: For things like camera, location, or storage access, the app must ask the user for permission first. We usually check if permission is already given, and if not, we request it. If the user says no, we should show a friendly message instead of crashing the app.

42. Crash Reporting (Crashlytics)

Answer: Tools like Firebase Crashlytics are added to the app so that if it crashes on a user's phone, we automatically get a report – what screen it happened on, what the error was, what device/OS version. This

helps us fix bugs we wouldn't otherwise know about, since we can't see the user's phone directly.

43. Class Lifecycle Methods vs Hook Equivalents

Answer: In class components, `componentDidMount` runs once when the screen loads, `componentDidUpdate` runs every time props/state change, and `componentWillUnmount` runs right before the component is removed, used for cleanup. With hooks, all of this is handled using `useEffect` — an empty array `[]` behaves like `componentDidMount`, returning a function inside `useEffect` behaves like `componentWillUnmount`, and adding values in the dependency array behaves like `componentDidUpdate` for those specific values.

44. AppState / App Lifecycle (Foreground, Background)

Answer: `AppState` tells us if the app is currently active (in use), in the background (minimized), or inactive. We use this to do things like pause a video when the user leaves the app, or refresh data when the user comes back to the app after a while.

```
AppState.addListener("change",
(nextState) => {
  if (nextState === "active")
    refreshData();
});
```

45. Error Boundary

Answer: An Error Boundary is a special component that catches JavaScript errors happening anywhere in its child components, so instead of the whole app crashing with a blank white screen, it shows a fallback UI like "Something went wrong." It only works for class components (using `componentDidCatch`), so even in a hooks-based app, this one part still needs a class component.

46. Animated API vs Reanimated

Answer: Both are used to create animations in React Native. The built-in `Animated` API works fine for simple animations, but it runs a lot of its logic on the JavaScript thread, which can lag if the app is busy. `Reanimated` (a separate library) runs animations directly on the native/UI thread, so they stay smooth even if JavaScript is busy doing something else —

that's why it's preferred for complex or gesture-based animations.

47. Important Libraries

Answer: Some libraries used in almost every React Native project: **React Navigation** for moving between screens, **Axios** for API calls, **Redux Toolkit** for managing app-wide data, **AsyncStorage** for simple local storage, **Firebase** for notifications and analytics, **WebView** to show a website inside the app, **Lottie** for animations, **React Native Maps** for showing maps, and **Razorpay/Stripe** for payments.

48. Flexbox & Responsive Design

Answer: React Native uses Flexbox to arrange things on the screen, just like CSS, but by default things are arranged in a column, not a row (this is different from web). The main properties are `flex`, `flexDirection`, `justifyContent`, and `alignItems`. For responsive design (different screen sizes), we avoid fixed pixel values and instead use percentages or the `useWindowDimensions` hook.

49. Testing (Jest & React Native Testing Library)

Answer: Jest is used to test plain functions and logic, like checking if a function gives the right output. React Native Testing Library is used to test components — like rendering a button and checking if pressing it does what it's supposed to do.

50. Build & Release (APK vs AAB)

Answer: APK is the file you install directly on an Android phone, mostly used for testing or sharing. AAB is the format the Play Store needs now — Google uses it to create a smaller, optimized version of the app for each user's phone. iOS apps are built and submitted differently, through Xcode as an `.ipa` file.

51. CI/CD (Fastlane & CodePush)

Answer: Fastlane is a tool that automates building and uploading the app to the Play Store/App Store, so you don't have to do it manually every time. CodePush lets you push small JavaScript-only updates (like bug fixes) directly to users without waiting for app store approval. But if you change native code, you still need a full release through the store.

52. Security (Token Storage & SSL Pinning)

Answer: Never save login tokens in plain AsyncStorage — always use Secure Storage (Keychain/Keystore) since it's encrypted. SSL Pinning is an extra layer of security where the app only trusts one specific server certificate, so even if someone tries to intercept the network traffic (like on a public Wi-Fi), the app will refuse to connect to anything fake.

[← Back to top](#)